

命令行个人工具箱项目需求文档

一、项目概述

本项目是为了巩固黑马 Python 基础阶段所有核心知识点设计的综合练习项目，将你提到的「字符串 / 文件工具集」和「持久化待办清单」两个小项目融合为一个完整的命令行工具箱。

通过完成本项目，你可以一次性练完零基础阶段的所有核心语法，包括：变量、数据类型、流程控制、函数、模块与包、文件操作、异常处理等，确保基础知识点无遗漏，为后续进阶学习打好基础。

二、项目整体目录结构

你需要按照以下结构创建项目文件，这部分用来练习**自定义包与模块**的知识点：

代码块

```
1  my_python_toolbox/      # 项目根目录
2  |—— main.py             # 主程序：命令行交互入口
3  |—— my_tools/           # 自定义工具包（必须创建这个文件夹）
4      |—— __init__.py      # 包标识文件：空文件即可，用来告诉Python这是一个包
5      |—— str_tools.py    # 字符串处理模块：存放所有字符串相关工具函数
6      |—— file_tools.py   # 文件操作模块：存放文件读写、异常处理的工具函数
7      |—— todo_tools.py   # 待办清单模块：存放待办的增删改查与持久化逻辑
```

三、分模块详细需求

3.1 字符串处理模块 str_tools.py

本模块用来练习**字符串操作**、**函数定义**、**参数与返回值**的知识点，你需要实现以下 3 个函数：

3.1.1 字符串反转函数 str_reverse(s)

- 功能：接收一个字符串参数，返回反转后的新字符串
- 示例：输入 `s="hello"`，返回 `"olleh"`
- 要求：不能直接使用 Python 内置的 `s[::-1]` 语法（要自己通过下标 / 循环实现，练基础逻辑）

3.1.2 自定义切片函数 str_slice(s, start, end, step=1)

- 功能：模拟 Python 内置的字符串切片功能，实现自定义的子串提取

- 参数：
 - `s`：待处理的原字符串
 - `start`：起始下标
 - `end`：结束下标（不包含）
 - `step`：步长，默认值为 1（练习默认参数知识点）
- 示例：输入 `s="hello world", start=0, end=5`，返回 `"hello"`
- 要求：自己通过循环和下标访问实现，不能直接用内置的 `s[start:end:step]`

3.1.3 字符统计函数 `str_count_char(s)`

- 功能：统计输入字符串中，英文字符、数字、空格、其他字符的数量
 - 返回值：返回一个包含统计结果的字典，格式如 `{"alpha": 5, "digit": 3, "space": 2, "other": 1}`
 - 练习点：字符串遍历、`for` 循环、字符串内置判断方法（`isalpha()` / `isdigit()` 等）
-

3.2 文件操作模块 `file_tools.py`

本模块用来练习**文件操作**、**异常处理**、**os 模块基础**的知识点，实现安全的文件操作工具，避免程序因为文件错误崩溃：

3.2.1 安全读文件函数 `safe_read_file(file_path, encoding="utf-8")`

- 功能：读取指定路径的文本文件，返回文件内容
- 异常处理：必须捕获以下异常，返回友好提示，不能让程序崩溃：
 - a. `FileNotFoundError`：文件不存在时，提示「文件不存在，请检查路径」
 - b. `PermissionError`：权限不足时，提示「没有权限读取该文件」
 - c. `UnicodeDecodeError`：文件编码错误时，提示「文件编码错误，无法读取为文本」
- 要求：使用 `with` 语句打开文件，自动关闭文件（练习 with 上下文管理器）

3.2.2 安全写文件函数 `safe_write_file(file_path, content, append=False)`

- 功能：将内容写入指定文件
- 参数：
 - `append`：是否为追加模式，默认 `False`（覆盖写入），为 `True` 时在文件末尾追加内容（练习默认参数）
- 异常处理：同样捕获权限错误、路径错误等异常，给出友好提示

3.2.3 文件检查函数 `check_file_exists(file_path)`

- 功能：检查指定路径的文件是否存在
 - 要求：使用 `os.path` 模块的方法实现（练习 `os` 模块的基础文件操作）
 - 返回值：存在返回 `True`，不存在返回 `False`
-

3.3 待办清单模块 `todo_tools.py`

本模块是核心综合练习，用来练习**列表**、**字典**、**函数封装**、**数据持久化**的知识点，实现带持久化的待办清单：

3.3.1 数据结构定义

- 用**列表**存储所有待办任务
- 列表中的每个元素是一个**字典**，用来存储单个任务的详情，格式如下：

代码块

```
1  # 示例任务
2  {
3      "task_id": 1,          # 任务唯一ID，数字类型
4      "content": "学习Python基础", # 任务内容
5      "is_finished": False # 任务状态：False未完成，True已完成
6  }
```

- 定义全局变量 `tasks` 用来存储当前的任务列表，程序启动时加载，退出时保存

3.3.2 持久化函数

实现任务的持久化，关闭程序后任务不会丢失，这部分要调用你刚才写的 `file_tools` 模块的函数，练习模块间的调用：

1. `load_tasks()`：
 - 程序启动时调用，从 `todo.txt` 文件中加载之前保存的任务
 - 如果文件不存在，返回空的任务列表
2. `save_tasks()`：
 - 每次修改任务后调用，把当前的 `tasks` 列表保存到 `todo.txt` 文件中

3.3.3 待办增删改查函数

你需要封装以下函数，实现待办的核心功能：

1. `add_task(content)`：

- 功能：添加新任务，自动生成唯一的 task_id（比如当前最大 ID+1，空列表从 1 开始）
 - 操作完成后自动调用 `save_tasks()` 保存数据
 - 返回：添加成功返回 True
2. `delete_task(task_id)`：
- 功能：根据 task_id 删除对应的任务
 - 处理：如果任务 ID 不存在，返回 False 提示用户
 - 操作完成后自动保存数据
3. `update_task(task_id, new_content=None, new_status=None)`：
- 功能：修改任务，支持修改任务内容、修改任务完成状态
 - 参数：`new_content` 和 `new_status` 默认值为 None（练习默认参数，用户可以只修改其中一个）
 - 处理：如果任务 ID 不存在，返回 False
 - 操作完成后自动保存数据
4. `query_tasks(keyword=None)`：
- 功能：查询任务，如果传入 keyword，就搜索内容包含 keyword 的任务；否则返回所有任务
 - 返回：符合条件的任务列表
5. `get_all_tasks()`：返回所有任务，给主程序调用展示用
-

四、主程序 `main.py` 需求

主程序是整个工具箱的交互入口，用来练习流程控制、循环嵌套、输入输出、模块导入的知识点，实现命令行菜单交互：

4.1 程序启动流程

1. 程序启动后，首先调用 `todo_tools.load_tasks()`，加载之前保存的待办任务
2. 进入主菜单循环，直到用户选择退出才结束

4.2 主菜单逻辑

循环展示以下主菜单，获取用户的选择：

代码块

```
1  ===== 个人命令行工具箱 =====
2  1. 字符串处理工具
3  2. 文件操作工具
```

```
4  3. 待办清单工具
5  0. 退出程序
6  请输入你的选择(0-3):
```

- 输入处理：如果用户输入的不是数字，或者输入的数字超出 0-3 的范围，要提示「输入无效，请重新输入」，不能让程序崩溃（练习异常处理）
- 根据用户的选择，进入对应的子菜单

4.3 字符串处理子菜单

用户选择 1 后，进入这个子菜单的循环，直到用户选 0 返回主菜单：

代码块

```
1  ===== 字符串处理工具 =====
2  1. 字符串反转
3  2. 自定义切片
4  3. 字符统计
5  0. 返回主菜单
6  请输入你的选择(0-3):
```

- 根据用户选择，调用 `str_tools` 里的对应函数：
 - 比如选 1：提示用户输入要处理的字符串，调用 `str_reverse`，然后输出结果
 - 选 2：提示用户输入起始、结束下标和步长，调用 `str_slice`，输出结果

4.4 文件操作子菜单

用户选择 2 后，进入这个子菜单：

代码块

```
1  ===== 文件操作工具 =====
2  1. 读取文件内容
3  2. 写入文件内容
4  3. 检查文件是否存在
5  0. 返回主菜单
6  请输入你的选择(0-3):
```

- 根据选择调用 `file_tools` 的对应函数，比如选 1：提示用户输入文件路径，调用 `safe_read_file`，输出内容或者错误提示

4.5 待办清单子菜单

用户选择 3 后，进入这个子菜单：

代码块

```
1  ===== 待办清单工具 =====
2  1. 查看所有待办
3  2. 添加新待办
4  3. 删除待办
5  4. 修改待办
6  5. 搜索待办
7  0. 返回主菜单
8  请输入你的选择(0-5):
```

- 根据选择调用 `todo_tools` 的对应函数：
 - 选 1：展示所有任务，格式要友好，比如： `[1] [未完成] 学习Python基础`
 - 选 2：提示用户输入任务内容，调用 `add_task` ，提示添加成功
 - 选 3：提示用户输入要删除的任务 ID，调用 `delete_task` ，提示结果
 - 以此类推，实现所有增删改查的交互

五、知识点覆盖对照表

本项目完整覆盖黑马 Python 基础阶段所有核心知识点，你可以对照检查自己的掌握情况：

黑马 Python 基础知识点	项目对应练习部分
环境搭建与 PyCharm 使用	项目目录创建、代码运行与调试
变量与输入输出	主程序的 input 输入、print 格式化输出、变量存储用户选择
基础数据类型（字符串、列表、字典）	字符串工具的字符串操作、待办的列表存任务、字典存任务详情
运算符	任务 ID 生成的算术运算、状态判断的比较运算符
流程控制（if 判断）	功能选择的 if-elif-else 分支、任务存在性判断
循环语句（while/for 循环）	

	主菜单的循环、子菜单的嵌套循环、字符串遍历的 for 循环
循环嵌套	主菜单与子菜单的双层循环结构
函数定义与调用	所有模块的功能函数、主程序调用模块函数
函数参数（位置参数、默认参数）	切片函数的默认步长、写文件的默认模式、修改任务的可选参数
函数返回值	所有工具函数的结果返回，比如读取文件的内容、操作的成功状态
列表推导式（可选）	查询任务时的列表筛选，可选择用推导式简化代码
模块与包	自定义 my_tools 包、模块划分、模块的导入与跨模块调用
文件操作	文件工具的读写、待办的持久化存储、with 上下文管理器
os 模块基础操作	文件存在性检查的 os.path 方法
异常处理	文件操作的异常捕获、用户输入的错误处理
字符串内置方法	字符统计的 isalpha/isdigit 方法、字符串的判断与处理
列表增删改查	待办任务的添加、删除、修改操作
字典的操作	任务详情的存储、键值的增删改查

六、开发步骤指引

建议你按照以下顺序开发，循序渐进巩固知识点：

1. **第一步：搭建目录：**先创建好所有的空文件，确认包结构正确

2. **第二步：实现 `str_tools`**：先写最简单的字符串工具，写完后可以单独测试每个函数
 3. **第三步：实现 `file_tools`**：然后写文件工具，重点测试异常处理，比如读一个不存在的文件看会不会崩溃
 4. **第四步：实现 `todo_tools`**：然后写待办模块，先写增删改查，再加上持久化，测试重启程序任务会不会丢失
 5. **第五步：实现 `[main.py](main.py)`**：最后写主程序，把所有模块串起来，实现菜单交互
 6. **第六步：整体测试**：测试所有功能，测试各种异常输入，确保程序不会崩溃
-

七、验收标准

完成后你可以按照以下标准检查自己的项目是否达标：

1. 目录结构完全符合要求，自定义包可以正常导入，没有导入错误
2. 所有功能都可以正常运行，没有逻辑错误
3. 程序不会崩溃：不管用户输入什么错误内容，还是文件出错，都只会给出友好提示，不会弹出 Traceback 报错
4. 待办任务可以持久化：关闭程序再重新打开，之前的任务还在
5. 所有基础知识点都有练习到，没有遗漏

（注：文档部分内容可能由 AI 生成）